Package ‘RMeDPower2’

September 24, 2025

Type Package

Title What the Package Does (Title Case)

Version 0.1.0

Author Min-Gyoung Shin and Reuben Thomas

Maintainer The package maintainer <yourself@somewhere.net>

Description More about what it does (maybe more than one line)
Use four spaces when indenting paragraphs within the Description.

License GPL-3

Encoding UTF-8

LazyData true

RoxygenNote 7.3.2

VignetteBuilder knitr

Depends R (>= 4.0), lme4, dplyr, simr, magrittr, ggplot2, ggtext,
quantreg

Imports tidyr, lmerTest, DHARMa, influence.ME, magick, EnvStats,
jsonlite, methods

Suggests knitr, rmarkdown, testthat (>= 3.0.0)

NeedsCompilation no

Contents

calculatePower *calculatePower*

Description

This functions makes statistical power estimates given the data, the underlying design for it and the assumed probability model of the error distribution

Usage

```
calculatePower(data, design, model, power_param)
```

Arguments

data	Input data frame with columns having all the necessary information regarding the dependent and independent variables of interest
design	an object of class RMeDesign with the necessary design information about the data
model	an object of class ProbabilityModel giving the error distribution of the data
power_param	an object of class PowerParams giving the target parameter of interest and the other necessary parameter to perform the power estimation

Value

A power curve as a ggplot object or a power calculation result printed in a text file

Examples

```
result=calculatePower(data=data, design=design, model=model, power_param=power_param)
```

calculate_lmer_estimates *calculate_lmer_estimates_covariate*

Description

This function performs a (generalized) linear mixed model analysis using (g)lmer.

Note: The current version does not accept categorical response variables, sample size parameters smaller than the observed samples size

Usage

```
calculate_lmer_estimates(
  data,
  condition_column,
  experimental_columns,
  response_column,
  total_column,
  condition_is_categorical,
```

```
  covariate = NULL,
  crossed_columns = NULL,
  error_is_non_normal = FALSE,
  family_p = NULL,
  na.action = "complete",
  include_interaction = FALSE,
  random_slope_variable = NULL,
  covariate_is_categorical = TRUE
)
```

Arguments

`data` Input data

`condition_column`
: Name of the condition variable (ex variable with values such as control/case). The input file has to have a corresponding column name

`experimental_columns`
: Name of variables related to experimental design such as "experiment", "plate", and "cell_line". They should be in order, for example, "experiment" should always come first .

`response_column`
: Name of the variable observed by performing the experiment. ex) intensity.

`total_column` Set this column only when family_p="binomial" and it is equal to the total number of observations (number of cases plus number of controls) for a given number of cases, when family_p="poisson" or "negative_binomial" and it is represents the total number of observations to be used as offset in the model

`condition_is_categorical`
: Specify whether the condition variable is categorical. TRUE: Categorical, FALSE: Continuous.

`covariate` The name of the covariate to control in the regression model

`crossed_columns`
: Name of experimental variables that may appear repeatedly with the same ID. For example, cell_line C1 may appear in multiple experiments, but plate P1 cannot appear in more than one experiment

`error_is_non_normal`
: Default: the observed variable is continuous Categorical response variable will be implemented in the future. TRUE: Categorical , FALSE: Continuous (default).

`family_p` The type of distribution family to specify when the response is categorical. If family is "binary" then binary(link="log") is used, if family is "poisson" then poisson(link="logit") is used, if family is "poisson_log" then poisson(link=") log") is used.

`na.action` "complete": missing data is not allowed in all columns (default), "unique": missing data is not allowed only in condition, experimental, and response columns. Selecting "complete" removes an entire row when there is one or more missing values, which may affect the distribution of other features.

`include_interaction`
: Whether to include condition * covariate interaction

`random_slope_variable`
: Variable for random slopes (typically "condition_column")

covariate_is_categorical
: Specify whether the covariate variable is categorical. TRUE: Categorical, FALSE: Continuous.

Value

A linear mixed model result

Examples

```
result=calculate_lmer_estimates(data=RMeDPower_data1,
condition_column="classification",
experimental_columns=c("experiment", "line"),
response_column="cell_size1",
condition_is_categorical=TRUE,
covariate="covariate",
crossed_columns = "line",
family_p=NULL,
error_is_non_normal=FALSE)
```

---

calculate_power *calculate_power_covariate*

Description

This function uses simulation to perform power analysis. It is designed to explore the power of biological experiments and to suggest an optimal number of experimental variables with reasonable power. The backbone of the function is based on simr package, which fits a fixed effect or mixed effect model based on the observed data and simulates response variables. Users can test the power of different combinations of experimental variables and parameters.

Note: The current version does not accept categorical response variables, sample size parameters smaller than the observed samples size

Usage

```
calculate_power(
  data,
  condition_column,
  experimental_columns,
  response_column,
  total_column = NULL,
  target_columns,
  power_curve,
  condition_is_categorical,
  covariate = NA,
  crossed_columns = NULL,
  error_is_non_normal = FALSE,
  nsimn = 1000,
  family_p = NULL,
  levels = NULL,
  max_size = NULL,
  breaks = NULL,
```

```
  effect_size = NULL,
  ICC = NULL,
  na.action = "complete",
  output = NULL,
  alpha = 0.05,
  include_interaction = NA,
  random_slope_variable = NULL,
  covariate_is_categorical = NA
)
```

Arguments

`data` Input data

`condition_column`
: The name of the condition variable (ex a variable with values such as control/case). The input file has to have a corresponding column name

`experimental_columns`
: Names of variables related to the experimental design, such as "experiment", "plate", and "cell_line".vThey should be in order, for example, "experiment" should always come first .

`response_column`
: The name of the variable observed by performing the experiment. ex) intensity.

`total_column` Set this column only when family_p="binomial" and it is equal to the total number of observations (number of cases plus number of controls) for a given number of cases

`target_columns` Name of the experimental parameters to use for the power calculation.

`power_curve` 1: Power simulation over a range of sample sizes or levels. 0: Power calculation over a single sample size or a level.

`condition_is_categorical`
: Specify whether the condition variable is categorical. TRUE: Categorical, FALSE: Continuous.

`covariate` The name of the covariate to control in the regression model

`crossed_columns`
: Name of experimental variables that may appear repeatedly with the same ID. For example, cell_line C1 may appear in multiple experiments, but plate P1 cannot appear in more than one experiment

`error_is_non_normal`
: Default: Observed variable is continuous. Categorical response variable will be implemented in the future. TRUE: Categorical , FALSE: Continuous (default).

`nsimn` The number of simulations to run. Default=1000

`family_p` The type of distribution family to specify when the response is categorical. If family is "binary" then binary(link="log") is used, if family is "poisson" then poisson(link="logit") is used, if family is "poisson_log" then poisson(link=") log") is used.

`levels` 1: Amplify the number of corresponding target parameter. 0: Amplify the number of samples from the corresponding target parameter, ex) If target_columns = c("experiment","cell_line") and if you want to expand the number of experiment and sample more cells from each cell line, set levels = c(1,0).

`max_size` Maximum levels or sample sizes to test. Default: the current level or the current sample size x 5. ex) If max_levels = c(10,5), it will test upto 10 experiments and 5 cell lines.

`breaks` Levels /sample sizes of the variable to be specified along the power curve. Default: max(1, round(the number of current levels / 5))

`effect_size` If you know the effect size of your condition variable, the effect size can be provided as a parameter. If the effect size is not provided, it will be estimated from your data

`ICC` Intra-Class Coefficients (ICC) for each parameter

`na.action` "complete": missing data is not allowed in all columns (default), "unique": missing data is not allowed only in condition, experimental, response, and target columns. Selecting "complete" removes an entire row when there is one or more missing values, which may affect the distribution of other features.

`output` Output file name

`alpha` Threshold for Type I error

`include_interaction` Whether to include condition * covariate interaction

`random_slope_variable` Variable for random slopes (typically "condition_column")

`covariate_is_categorical` Specify whether the covariate variable is categorical. TRUE: Categorical, FALSE: Continuous.

Value

A power curve as a ggplot object or a power calculation result printed in a text file

Examples

```
result=calculate_power(data=RMeDPower_data1,
condition_column="classification",
experimental_columns=c("experiment", "line"),
response_column="cell_size1",
target_columns="experiment",
power_curve=1,
condition_is_categorical=TRUE,
crossed_columns = "line",
error_is_non_normal=FALSE,
levels=1)
```

`diagnoseDataModel` *diagnoseDataModel*

Description

This function can be used to generate diagnostic QC plots for given model assumptions related to the input data, identify potential outlier observations and/or outlier experimental units

Usage

```
diagnoseDataModel(data, design, model)
```

Arguments

`data`	Input data frame with columns having all the necessary information regarding the dependent and independent variables of interest
`design`	an object of class RMeDesign with the necessary design information about the data
`model`	an object of class ProbabilityModel giving the error distribution of the data

Value

A list with four elements. 1) models: representing the names of the models evaluated based on differnt modifications of the response column. The models would include one called natural_scale, another model called natural_scale_wo_outliers if outliers had beeen identified, another model called log_scale if the respose column is continuous and the model on the log-transformed values of the responses are what was evaluated and finally log_scale_wo_outliers model if there were outliers identified in the log_scale model. 2) Data_updated representing the updated data frame with additional columns for the modified response column corresponding to each of the models evaluated. 3) cooks_result: cooks distance of each of the experimental columns for each of the models evaluated. For models based on the binomial probability distribution, cooks distance is only reported for the first experimental column on account the increased computation time for evaluating this metric for the other experimental columns. 4) plots_info: is a list with two elements plots and captions. plots is a named list and captions is a character vector, both of the same length as the number of models evaluated. Each element of the plots list is yet another list of QC/diagnostic plots related to the corresponding model fit, while the captions is a vector of captions for each of the QC plots output

Examples

```
result=diagnoseDataModel(data=data, design=design, model=model)
```

`getEstimatesOfInterest`

getEstimatesOfInterest

Description

This function performs the estimations of interest and also visualizes the resulting association

Usage

```
getEstimatesOfInterest(data, design, model, print_plots = TRUE)
```

Arguments

`data`	Input data frame with columns having all the necessary information regarding the dependent and independent variables of interest
`design`	an object of class RMeDesign with the necessary design information about the data
`model`	an object of class ProbabilityModel giving the error distribution of the data
`print_plots`	Whether or not to print the plots, irrespective of this argument ggplot versions of evaluated association between the response_column and the condition_column. TRUE - print the plot, FALSE - do not print the plot

Value

a list with two elements - 1. an object of class summary.merMod and 2. the output from the get_residuals functions. This output consists of a list with 3 elements. 1. The updated input data with an additional column with the model residuals of the individual observations. 2. A plot representing the purported association between the response column and the condition column. 3. The corresponding caption for this figure.

Examples

```
result=diagnoseDataModel(data=data, design=design, model=model)
```

---

get_model_and_data *get_model_and_data*

Description

This function performs a linear mixed model analysis using lmer.

Usage

```
get_model_and_data(
  data,
  condition_column,
  experimental_columns,
  response_column,
  total_column = NULL,
  condition_is_categorical,
  covariate = NULL,
  crossed_columns = NULL,
  error_is_non_normal = FALSE,
  family_p = NULL,
  na.action = "complete",
  include_interaction = FALSE,
  random_slope_variable = NULL,
  covariate_is_categorical = TRUE
)
```

Arguments

`data` Input data

`condition_column`
: Name of the condition variable (ex variable with values such as control/case). The input file has to have a corresponding column name

`experimental_columns`
: Name of variables related to experimental design such as "experiment", "plate", and "cell_line". They should be in order, for example, "experiment" should always come first .

`response_column`
: Name of the variable observed by performing the experiment. ex) intensity.

`total_column` Set this column only when family_p="binomial" and it is equal to the total number of observations (number of cases plus number of controls) for a given number of cases

`condition_is_categorical`
Specify whether the condition variable is categorical. TRUE: Categorical, FALSE: Continuous.

`covariate` The name of the covariate to control in the regression model

`crossed_columns`
Name of experimental variables that may appear repeatedly with the same ID. For example, cell_line C1 may appear in multiple experiments, but plate P1 cannot appear in more than one experiment

`error_is_non_normal`
Default: the observed variable is continuous Categorical response variable will be implemented in the future. TRUE: Categorical , FALSE: Continuous (default).

`family_p` The type of distribution family to specify when the response is categorical. If family is "binary" then binary(link="log") is used, if family is "poisson" then poisson(link="logit") is used, if family is "poisson_log" then poisson(link=") log") is used.

`na.action` "complete": missing data is not allowed in all columns (default), "unique": missing data is not allowed only in condition, experimental, and response columns. Selecting "complete" removes an entire row when there is one or more missing values, which may affect the distribution of other features.

`include_interaction`
Whether to include condition * covariate interaction

`random_slope_variable`
Variable for random slopes (typically "condition_column")

`covariate_is_categorical`
Specify whether the covariate variable is categorical. TRUE: Categorical, FALSE: Continuous.

Value

A list of the linear mixed model result, original data, experimental column names, and residual values

---

`get_residuals` *get_residuals_covariate*

Description

This function retrieve residual values from an lmerFit summary object and plot residual values by condition_column

Note: The current version does not accept categorical response variables, sample size parameters smaller than the observed samples size

Usage

```
get_residuals(
  data,
  condition_column,
  experimental_columns,
  response_column,
  condition_is_categorical,
  covariate = NULL,
  crossed_columns = NULL,
  total_column = FALSE,
  error_is_non_normal = FALSE,
  family_p = NULL,
  na.action = "complete",
  include_interaction = FALSE,
  random_slope_variable = NULL,
  covariate_is_categorical = TRUE,
  print_plots = TRUE
)
```

Arguments

`data` Input data

`condition_column`
Name of the condition variable (ex variable with values such as control/case). The input file has to have a corresponding column name

`experimental_columns`
Name of variables related to experimental design such as "experiment", "plate", and "cell_line". They should be in order, for example, "experiment" should always come first .

`response_column`
Name of the variable observed by performing the experiment. ex) intensity.

`condition_is_categorical`
Specify whether the condition variable is categorical. TRUE: Categorical, FALSE: Continuous.

`covariate` The name of the covariate to control in the regression model

`crossed_columns`
Name of experimental variables that may appear repeatedly with the same ID. For example, cell_line C1 may appear in multiple experiments, but plate P1 cannot appear in more than one experiment

`total_column` Set this column only when family_p="binomial" and it is equal to the total number of observations (number of cases plus number of controls) for a given number of cases

`error_is_non_normal`
Default: the observed variable is continuous Categorical response variable will be implemented in the future. TRUE: Categorical , FALSE: Continuous (default).

`family_p` The type of distribution family to specify when the response is categorical. If family is "binary" then binary(link="log") is used, if family is "poisson" then poisson(link="logit") is used, if family is "poisson_log" then poisson(link=") log") is used.

`na.action` "complete": missing data is not allowed in all columns (default), "unique": missing data is not allowed only in condition, experimental, and response columns. Selecting "complete" removes an entire row when there is one or more missing values, which may affect the distribution of other features.

`include_interaction` Whether to include condition * covariate interaction

`random_slope_variable` Variable for random slopes (typically "condition_column")

`covariate_is_categorical` Specify whether the covariate variable is categorical. TRUE: Categorical, FALSE: Continuous.

`print_plots` Whether or not to print the plots, irrespective of this argument ggplot versions of evaluated association between the response_column and the condition_column. TRUE - print the plot, FALSE - do not print the plot

Value

A list with 3 elements. 1. The updated input data with an additional column with the model residuals of the individual observations. 2. A plot representing the purported association between the response column and the condition column. 3. The corresponding caption for this figure.

Examples

```
result=get_residuals_covariate(data=RMeDPower_data1,
condition_column="classification",
experimental_columns=c("experiment", "line"),
response_column="cell_size1",
condition_is_categorical=TRUE,
covariate="covariate",
crossed_columns = "line",
error_is_non_normal=FALSE)
```

PowerParams-class *PowerParams-class*

Description

Objects of PowerParams class store information required for sample size estimation for given data

Arguments

`power_curve` 1: Power simulation over a range of sample sizes or levels. 0: Power calculation over a single sample size or a level.

`nsimn` The number of simulations to run. Default=1000

`target_columns` Name of the experimental parameters to use for the power calculation.

`levels` 1: Amplify the number of corresponding target parameter. 0: Amplify the number of samples from the corresponding target parameter, ex) If target_columns = c("experiment","cell_line") and if you want to expand the number of experiment and sample more cells from each cell line, set levels = c(1,0).

`max_size` Maximum levels or sample sizes to test. Default: the current level or the current sample size x 5. ex) If max_levels = c(10,5), it will test upto 10 experiments and 5 cell lines.

`breaks` Levels /sample sizes of the variable to be specified along the power curve. Default: max(1, round(the number of current levels / 5))

`effect_size` If you know the effect size of your condition variable, the effect size can be provided as a parameter. If the effect size is not provided, it will be estimated from your data

`alpha` Threshold for Type I error

`ICC` Intra-Class Coefficients (ICC) for each parameter

Value

an object of class ProbabilityModel

Examples

```
power_param=new("PowerParams")
```

`ProbabilityModel-class` *ProbabilityModel-class*

Description

Objects of ProbabilityModel class store information on the assumed probability distribution for the model

Arguments

`error_is_non_normal` Default: the observed variable is continuous Categorical response variable will be implemented in the future. TRUE: Categorical , FALSE: Continuous (default).

`family_p` The type of distribution family to specify when the response is categorical. If family is "binary" then binary(link="log") is used, if family is "poisson" then poisson(link="logit") is used, if family is "poisson_log" then poisson(link=") log") is used.

Value

an object of class ProbabilityModel

Examples

```
model=new("ProbabilityModel")
```

readDesign *readDesign*

Description

This functions reads the underlying design for the data

Usage

```
readDesign(jsonfile)
```

Arguments

`jsonfile`	the jsonfile with the necessary design parameters: condition_column, experimental_columns, response_column, total_column, condition_is_categorical, covariate, method, crossed_columns, error_is_non_normal, family_p, outlier_alpha, na.action

Value

an object of class RMeDesign

Examples

```
design=readDesign(jsonfile)
```

readPowerParams *readPowerParams*

Description

This functions reads the underlying design for the data

Usage

```
readPowerParams(jsonfile)
```

Arguments

`jsonfile`	the jsonfile with the necessary parameters for statistical power estimation: target_columns, power_curve, nsimn, levels, max_size, alpha, breaks, effect_size, icc

Value

an object of class PowerParams

Examples

```
design=readPowerParams(jsonfile)
```

readProbabilityModel *readProbabilityModel*

Description

This functions reads the underlying design for the data

Usage

```
readProbabilityModel(jsonfile)
```

Arguments

`jsonfile`	the jsonfile with the necessary parameters for probability model: error_is_non_normal, family_p

Value

an object of class ProbabilityModel

Examples

```
design=readPowerParams(jsonfile)
```

RMeDesign-class *RMeDesign-class*

Description

Objects of RMeDesign class store information on the relevant repeated measures design for the given data

Arguments

`data`	Input data
`condition_column`	Name of the condition variable (ex variable with values such as control/case). The input file has to have a corresponding column name
`experimental_columns`	Name of the variable related to experimental design such as "experiment", "plate", and "cell_line". They should be in order, for example, "experiment" should always come first .
`response_column`	Name of the variable observed by performing the experiment. ex) intensity.
`total_column`	Set this column only when family_p="binomial" and it is equal to the total number of observations (number of cases plus number of controls) for a given number of cases
`outlier_alpha`	numeric scalar between 0 and 1 indicating the Type I error associated with the test of outliers

`condition_is_categorical`
: Specify whether the condition variable is categorical. TRUE: Categorical, FALSE: Continuous.

`covariate`
: The name of the covariate to control in the regression model

`method`
: The method used to detect outliers. "rosner" (default) runs Rosner's test and "cook" runs Cook's distance.

`crossed_columns`
: Name of experimental variables that may appear repeatedly with the same ID. For example, cell_line C1 may appear in multiple experiments, but plate P1 cannot appear in more than one experiment

`error_is_non_normal`
: Default: the observed variable is continuous Categorical response variable will be implemented in the future. TRUE: Categorical , FALSE: Continuous (default).

`family_p`
: The type of distribution family to specify when the response is categorical. If family is "binary" then binary(link="log") is used, if family is "poisson" then poisson(link="logit") is used, if family is "poisson_log" then poisson(link=") log") is used.

`na.action`
: "complete": missing data is not allowed in all columns (default), "unique": missing data is not allowed only in condition, experimental, and response columns. Selecting "complete" removes an entire row when there is one or more missing values, which may affect the distribution of other features.

`include_interaction`
: logical - TRUE or FALSE - Whether to include condition * covariate interaction

`random_slope_variable`
: Variable for random slopes (typically "condition_column")

`covariate_is_categorical`
: Specify whether the covariate variable is categorical. TRUE: Categorical, FALSE: Continuous.

Value

an object of class RMeDesign

Examples

```
design=new("RMeDesign")
```

`transform_data` *transform_data*

Description

This function can be used to generate diagnostic QC plots for given model assumptions related to the input data, identify potential outlier observations and/or outlier experimental units

Usage

```
transform_data(
  data,
  condition_column,
  experimental_columns,
  response_column,
  total_column = NULL,
  condition_is_categorical = TRUE,
  covariate = NULL,
  crossed_columns = NULL,
  error_is_non_normal = FALSE,
  family_p = NULL,
  alpha = 0.05,
  na.action = "complete",
  include_interaction = NA,
  random_slope_variable = NULL,
  covariate_is_categorical = NA
)
```

Arguments

`data` Input data

`condition_column`
Name of the condition variable (ex variable with values such as control/case). The input file has to have a corresponding column name

`experimental_columns`
Name of the variable related to experimental design such as "experiment", "plate", and "cell_line". They should be in order, for example, "experiment" should always come first .

`response_column`
Name of the variable observed by performing the experiment. ex) intensity.

`total_column` Set this column only when family_p="binomial" and it is equal to the total number of observations (number of cases plus number of controls) for a given number of cases

`condition_is_categorical`
Specify whether the condition variable is categorical. TRUE: Categorical, FALSE: Continuous.

`covariate` The name of the covariate to control in the regression model

`crossed_columns`
Name of experimental variables that may appear repeatedly with the same ID. For example, cell_line C1 may appear in multiple experiments, but plate P1 cannot appear in more than one experiment

`error_is_non_normal`
Default: the observed variable is continuous Categorical response variable will be implemented in the future. TRUE: Categorical , FALSE: Continuous (default).

`family_p` The type of distribution family to specify when the response is categorical. If family is "binary" then binary(link="log") is used, if family is "poisson" then poisson(link="logit") is used, if family is "poisson_log" then poisson(link=") log") is used.

alpha
: numeric scalar between 0 and 1 indicating the Type I error associated with the test of outliers

na.action
: "complete": missing data is not allowed in all columns (default), "unique": missing data is not allowed only in condition, experimental, and response columns. Selecting "complete" removes an entire row when there is one or more missing values, which may affect the distribution of other features.

include_interaction
: logical - TRUE or FALSE - Whether to include condition * covariate interaction

random_slope_variable
: Variable for random slopes (typically "condition_column")

covariate_is_categorical
: Specify whether the covariate variable is categorical. TRUE: Categorical, FALSE: Continuous.

Value

A list with four elements. 1) models: representing the names of the models evaluated based on differnt modifications of the response column. The models would include one called natural_scale, another model called natural_scale_wo_outliers if outliers had beeen identified, another model called log_scale if the respose column is continuous and the model on the log-transformed values of the responses are what was evaluated and finally log_scale_wo_outliers model if there were outliers identified in the log_scale model. 2) Data_updated representing the updated data frame with additional columns for the modified response column corresponding to each of the models evaluated. 3) cooks_result: cooks distance of each of the experimental columns for each of the models evaluated. For models based on the binomial probability distribution, cooks distance is only reported for the first experimental column on account the increased computation time for evaluating this metric for the other experimental columns. 4) diagnostic_plots: is a list with two elements plots and captions. plots is a named list and captions is a character vector, both of the same length as the number of models evaluated. Each element of the plots list is yet another list of QC/diagnostic plots related to the corresponding model fit, while the captions is a vector of captions for each of the QC plots output 5) cooks_plots: is a list of plots is named list of the same length as the number of models evaluated. Each element of the cooks_plot is another list of bar plots of cooks distance plots for each of the experimental columns. The title of the plot indicate the model and experimental factor while the subtitle indicates the identified outliers if any using the 4/n threshold

Examples

```
result=transform_data2(data=data, condition_column="classif", experimental_columns=c("experiment","line"), 
result=transform_data2(data=data, condition_column="classif", experimental_columns=c("experiment","line"), 
result=transform_data2(data=data, condition_column="classif", experimental_columns=c("experiment","line"), 
```

Index